# SOPORTE TEMÁTICO

| **Nombre del tema 1:** | Python para analisis de datos |
|---|---|
| **Desarrollo del tema 1:** | |

## ¡Hola coder, ahora vamos a empezar a trabajar con los datos de verdad!

Hasta este punto has construido una base que es más importante de lo que parece.

Entendiste que los datos no son solo números, sino que siguen un proceso:

- Nacen en sistemas operacionales (OLTP)
- Se transforman y organizan
- Se analizan en entornos analíticos (OLAP)
- Y finalmente se convierten en decisiones

También viste que para poder analizar correctamente, los datos deben estar bien estructurados a través del modelado.

Ahora surge una pregunta natural:

Si ya entendemos cómo funcionan los datos… ¿cómo empezamos a trabajar con ellos?

## ¿Por qué Python en analítica de datos?

Python es uno de los lenguajes más utilizados en el mundo de los datos porque combina algo muy importante: es fácil de aprender, pero al mismo tiempo es extremadamente poderoso.

En analítica, Python te permite trabajar directamente con los datos: limpiarlos, transformarlos, analizarlos y prepararlos para obtener información útil. Sin embargo, su alcance va mucho más allá.

Python es un lenguaje versátil que se utiliza en múltiples áreas de la tecnología, lo que lo convierte en una herramienta muy valiosa a largo plazo. Con Python no solo puedes analizar datos, también puedes:

- automatizar tareas repetitivas
- consumir y crear APIs
- trabajar con bases de datos
- desarrollar aplicaciones web

- procesar archivos (Excel, CSV, JSON, etc.)
- construir modelos de machine learning
- trabajar con inteligencia artificial

Esto significa que al aprender Python, no solo estás adquiriendo una herramienta para analítica, sino una base que te puede abrir múltiples caminos dentro del mundo tecnológico.

## Instalación del entorno de trabajo

Antes de trabajar con datos, es necesario entender cómo se gestiona un entorno de trabajo profesional.

En proyectos reales, no se trabaja directamente sobre el sistema global. Cada proyecto tiene sus propias dependencias, versiones de librerías y configuraciones. Si no se controla esto, se generan conflictos.

**¿Como instalar UV?**

*Githubusercontent.com*

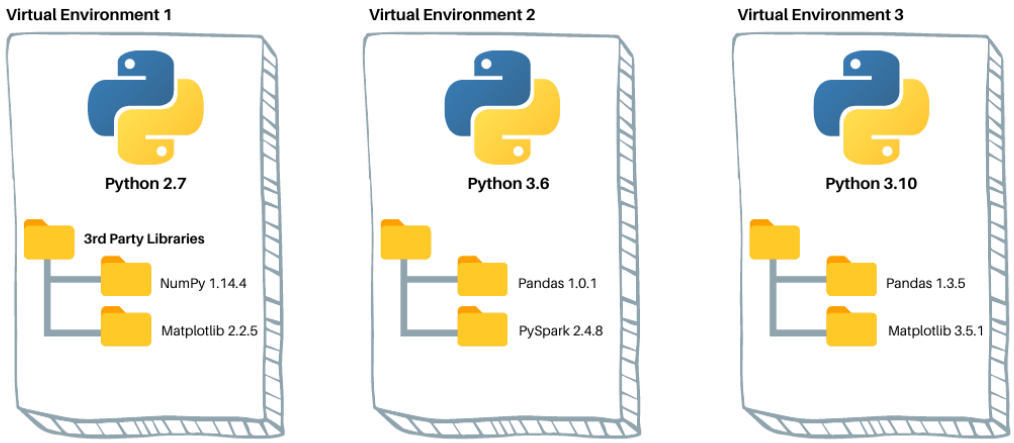
Para realizar la instalación de UV vamos hacer uso de la documentación oficial y seguir el paso a paso que vemos dentro de la pagina de UV Astral

**Guia para instalar UV Astral**

Luego de instalar UV vamos a hacer la instalación de la última versión disponible de Python

**Guía para instalar Python con UV Astral**

**¿Qué es un Environment en Python?**




Un **environment (entorno virtual)** es un espacio aislado donde puedes instalar y gestionar las librerías de un proyecto sin afectar el resto del sistema.

En otras palabras, es como crear una “caja independiente” para cada proyecto donde defines:

- qué versión de Python usar
- qué librerías instalar
- qué dependencias manejar

Esto es fundamental en analítica de datos porque cada proyecto puede necesitar configuraciones diferentes.

### ¿Por qué usar environments?

En el mundo real, no trabajas con un solo proyecto.

Imagina este escenario:

- Proyecto A usa Pandas 1.x
- Proyecto B usa Pandas 2.x

Si todo está instalado globalmente, tendrás conflictos.

Un environment resuelve esto porque:

- cada proyecto es independiente
- no hay conflictos de versiones
- puedes replicar el entorno fácilmente

Esto es estándar en cualquier entorno profesional.

### ¿Cómo instalar y usar environments con UV?

UV es una herramienta moderna que simplifica la gestión de entornos y dependencias en Python.

Guia para crear y manejar environments con UV

## Manipulación de estructuras de datos: la base de todo

Una vez tienes el entorno listo, el siguiente paso es entender cómo viven los datos dentro de Python.

Los datos no existen en abstracto dentro del código. Siempre están contenidos dentro de estructuras.

Las estructuras más comunes son listas y diccionarios. Las listas permiten trabajar con colecciones de elementos, mientras que los diccionarios permiten representar datos estructurados mediante claves y valores.

En analítica, estas estructuras suelen representar datasets en una forma básica. Por ejemplo, una lista puede representar una colección de valores, y un diccionario puede representar una fila de datos con múltiples atributos.

A medida que avanzas, empiezas a trabajar con estructuras más complejas, como listas de diccionarios, que se asemejan a datasets reales. Este es un punto clave porque aquí comienzas a simular cómo se ven los datos antes de usar herramientas más avanzadas.

### Ejemplo: lista de datos

```
    # Array con valores de ventas
ventas = [100, 200, 150]

# Iterar para mostrar cada dato del array de ventas
for venta in ventas:
    print(venta)

```

### Ejemplo: diccionario como registro:

```python
# Diccionario de un registro de cliente
cliente = {
    "nombre": "Juan",
    "ciudad": "Medellín",
    "edad": 30
}

# Mostrar la ciudad del cliente
print(cliente["ciudad"])
```

### Ejemplo: diccionario como registro

```python
# Ejemplo de un dataset de clientes
datos = [
    {"cliente": "Juan", "ventas": 100},
    {"cliente": "Ana", "ventas": 200}
]
```

## Comprensiones y operaciones avanzadas

Al inicio, es común usar loops para procesar datos. Sin embargo, Python permite una forma más eficiente mediante comprensiones.

Esto cambia la forma de trabajar: pasas de recorrer datos a transformarlos directamente.

### Ejemplo: forma tradicional

```python
# Datos simulados de ventas
ventas = [100, 200, 150]

# Lista vacía de datos
resultado = []

# Recorrer datos y validar para saber si agregar
for v in ventas:
    if v > 150:
        resultado.append(v)
```

**Ejemplo: comprensión**

```python
# Datos simulados de ventas
ventas = [100, 200, 150]

# Compresión de datos
ventas_mayores = [v for v in ventas if v > 150]
```

## Manejo de archivos: conexión con la realidad

Hasta ahora has trabajado con datos definidos dentro del código, pero en la realidad los datos vienen de fuentes externas.

La forma más común de recibir datos es a través de archivos. Los formatos más utilizados en analítica son CSV, Excel y archivos de texto.

Aprender a leer y escribir estos archivos es fundamental porque es el primer punto de contacto con datos reales. Aquí el estudiante entiende cómo cargar información, procesarla y generar nuevos archivos con resultados.

Este paso conecta directamente con el mundo empresarial, donde la mayoría de los datos iniciales vienen en este tipo de formatos.

**Ejemplo: lectura de CSV**

```python
# Forma de manipular archivos .CSV
with open("ventas.csv", "r") as file:
    for linea in file:
        print(linea)
```

**Ejemplo: lectura de TXT**

```python
# Manipular archivos .TXT
with open("log.txt", "r") as file:
    contenido = file.read()
    print(contenido)

```

## Procesamiento de datos con Python

Este es uno de los puntos más importantes de toda la clase.

Antes de introducir herramientas avanzadas, es necesario que el estudiante entienda qué significa procesar datos.

Procesar datos implica recorrer estructuras, aplicar condiciones, transformar valores y calcular métricas. Todo esto se puede hacer con Python puro.

Aquí el estudiante experimenta algo clave: trabajar con datos de esta forma puede volverse complejo y poco eficiente a medida que el volumen crece.

Y eso es precisamente el objetivo.

Porque cuando más adelante vea Pandas, entenderá no solo cómo usarlo, sino por qué existe.

### Ejemplo: cálculo total

```python
# Lista de diccionarios, cada uno representa a un cliente y su valor de ventas
datos = [
    {"cliente": "Juan", "ventas": 100},
    {"cliente": "Ana", "ventas": 200},
    {"cliente": "Pedro", "ventas": 150},
]

# Variable que acumulará la suma de todas las ventas
total = 0

# Bucle que recorre cada diccionario en la lista 'datos'
for d in datos:
    # Accede al valor asociado a la clave "ventas" y lo suma a 'total'
    total += d["ventas"]

# Muestra el resultado final en la pantalla
print("Total ventas:", total)
```

### Ejemplo: filtrado

```python
# Lista de diccionarios, cada uno representa a un cliente y su valor de ventas
datos = [
    {"cliente": "Juan", "ventas": 100},
    {"cliente": "Ana", "ventas": 200},
    {"cliente": "Pedro", "ventas": 150},
]
# Crear una nueva lista llamada 'clientes_top'
# con todos los elementos de 'datos' donde la venta es mayor que 150
clientes_top = [d for d in datos if d["ventas"] > 150]

# Mostrar el resultado (lista de diccionarios que cumplen la condición)
print(clientes_top)
```

## Manejo de errores y calidad de datos

En la teoría, los datos son perfectos. En la práctica, no lo son.

Los datos reales pueden contener errores, valores inesperados, formatos incorrectos o información incompleta. Por eso, el manejo de errores no es opcional, es fundamental.

Aquí se introduce el uso de mecanismos como try y except, que permiten controlar fallos en el código. Pero más allá de la técnica, lo importante es el concepto:

El analista debe asumir que los datos pueden fallar y debe prepararse para manejar esos casos.

Este punto conecta directamente con lo que verás en la siguiente semana sobre limpieza de datos.

### Ejemplo: manejo de errores

```python
# Lista de cadenas; algunas representan números, otras no
datos = ["100", "200", "abc", "300"]

# Bucle que recorre cada elemento de la lista 'datos'
for d in datos:
    try:
        # Intenta convertir el valor de la cadena 'd' a entero
        valor = int(d)
        # Si tiene éxito, imprime el número convertido
        print(valor)
    except:
        # Si ocurre un error (por ejemplo, no se puede convertir a entero),
        # imprime un mensaje indicando que el dato es inválido
        print("Dato inválido:", d)
```

## Eficiencia en el procesamiento de datos

Finalmente, aparece un concepto más avanzado: la eficiencia.

No basta con que el código funcione. En analítica, los volúmenes de datos pueden ser grandes, y una mala implementación puede afectar el rendimiento.

Aquí se introducen ideas como evitar operaciones innecesarias, elegir estructuras adecuadas y pensar en cómo escalar el procesamiento.

Este es el primer acercamiento a un pensamiento más técnico, donde no solo importa el resultado, sino cómo se obtiene.

**Ejemplo: Código ineficiente:**

```python
# Lista de diccionarios: cada cliente tiene nombre, ventas y estado 'activo'
datos = [
    {"cliente": "Juan", "ventas": 100, "activo": True},
    {"cliente": "Ana", "ventas": 200, "activo": False},
    {"cliente": "Pedro", "ventas": 150, "activo": True},
    {"cliente": "Luisa", "ventas": 300, "activo": True},
]

# Lista vacía para almacenar solo los clientes que están activos
activos = []

# Bucle que recorre cada diccionario en 'datos'
for d in datos:
    # Si el campo 'activo' es True, agrega ese diccionario a la lista 'activos'
    if d["activo"] == True:
        activos.append(d)

# Lista vacía para guardar solo los valores de ventas de los activos
ventas = []

# Bucle que recorre cada cliente activo
for a in activos:
    # Extrae el valor de 'ventas' y lo agrega a la lista 'ventas'
    ventas.append(a["ventas"])

# Variable que acumulará la suma de las ventas
total = 0

# Bucle que recorre cada valor en la lista 'ventas'
for v in ventas:
    # Suma cada valor al acumulador 'total'
    total += v

# Muestra el total final de ventas de los clientes activos
print("Total:", total)
```

**Problemas:**

- Múltiples loops innecesarios
- Uso de listas intermedias que no aportan valor
- Código más largo y difícil de mantener

**Ejemplo: Código eficiente**

```python
# Lista de diccionarios: cada cliente tiene nombre, ventas y estado 'activo'
datos = [
    {"cliente": "Juan", "ventas": 100, "activo": True},
    {"cliente": "Ana", "ventas": 200, "activo": False},
    {"cliente": "Pedro", "ventas": 150, "activo": True},
    {"cliente": "Luisa", "ventas": 300, "activo": True},
]

# Codigo eficiente
total = sum(d["ventas"] for d in datos if d["activo"])

print("Total:", total)
```

**Qué cambió realmente:**

- Eliminaste estructuras intermedias
- Redujiste múltiples recorridos a uno solo
- Expresaste la lógica de forma directa

**Conclusiones del tema 1:**

En esta lección diste un paso fundamental en tu formación como analista de datos: pasaste de entender los datos a empezar a trabajar directamente con ellos.

Aprendiste que Python no es solo un lenguaje de programación, sino una herramienta que te permite manipular, transformar y procesar información de forma flexible. Comprendiste cómo estructurar datos dentro del código, cómo leerlos desde archivos reales y cómo aplicar lógica para obtener resultados.

También viste algo muy importante: trabajar con datos en Python puro es posible, pero implica esfuerzo, múltiples pasos y, en muchos casos, soluciones poco eficientes. Esto no es un problema, es parte del aprendizaje, porque te permite entender realmente qué significa procesar datos.

Además, comenzaste a desarrollar criterio técnico al enfrentarte a situaciones reales como errores en los datos, valores inconsistentes y la necesidad de optimizar tu código.

En este punto ya no eres solo alguien que entiende qué son los datos, sino alguien que puede empezar a trabajarlos de forma estructurada.

| | |
|---|---|
| **Nombre del tema 2:** | Pandas |
| **Desarrollo del tema 2:** | |

**¡Hola coder, ahora vamos a empezar a trabajar con los datos como realmente se hace en el mundo profesional!**

Hasta este punto has recorrido un camino muy importante. No solo aprendiste qué son los datos y cómo se transforman en información y conocimiento, sino que también diste tus primeros pasos trabajando con ellos usando Python.

Ya sabes leer archivos, manipular estructuras como listas y diccionarios, recorrer datos, aplicar condiciones y calcular métricas. Es decir, ya tienes la base para trabajar con datos de forma técnica.

Pero también te enfrentaste a algo clave.

Trabajar con datos usando Python puro, aunque es posible, empieza a volverse complejo a medida que el volumen crece o las operaciones se hacen más elaboradas. Para responder una pregunta relativamente simple, muchas veces necesitas múltiples pasos, estructuras intermedias y bastante código.

Y aquí aparece una idea fundamental en tu formación:

**No se trata solo de poder hacerlo, sino de hacerlo de forma eficiente, clara y escalable.**

En el mundo real, los analistas no trabajan recorriendo listas manualmente ni escribiendo múltiples loops para cada cálculo. Trabajan con herramientas diseñadas específicamente para manipular datos de forma estructurada.

Aquí es donde entra Pandas.

## ¿Qué es Pandas?

Pandas es una librería de Python diseñada para trabajar con datos estructurados de manera eficiente. Su propósito no es simplemente almacenar datos, sino permitirte manipularlos, transformarlos y analizarlos de una forma mucho más natural.

Pero más allá de la definición técnica, lo importante es entender esto:

**Pandas no es solo una herramienta, es un cambio en la forma de trabajar con datos.**

Con Python puro, tú controlas cada paso del proceso. En Pandas, defines qué quieres hacer y la librería se encarga del cómo.

Esto significa que pasas de escribir código detallado paso a paso, a trabajar con operaciones más declarativas, más cercanas a cómo piensas el análisis.

Además, Pandas está completamente alineado con el mundo real. Las estructuras que utiliza son equivalentes a lo que ya conoces:

- tablas en bases de datos
- hojas de Excel
- datasets en analítica

Por eso, aprender Pandas no solo te enseña una librería, te acerca a cómo se manejan los datos en una empresa.

## Instalación y primeros pasos

Antes de empezar a trabajar con Pandas, es importante hacerlo como se hace en un entorno profesional: dentro de un proyecto organizado y con un entorno controlado.

En la práctica, no trabajas archivos sueltos. Trabajas dentro de proyectos que tienen su propia configuración, dependencias y estructura.

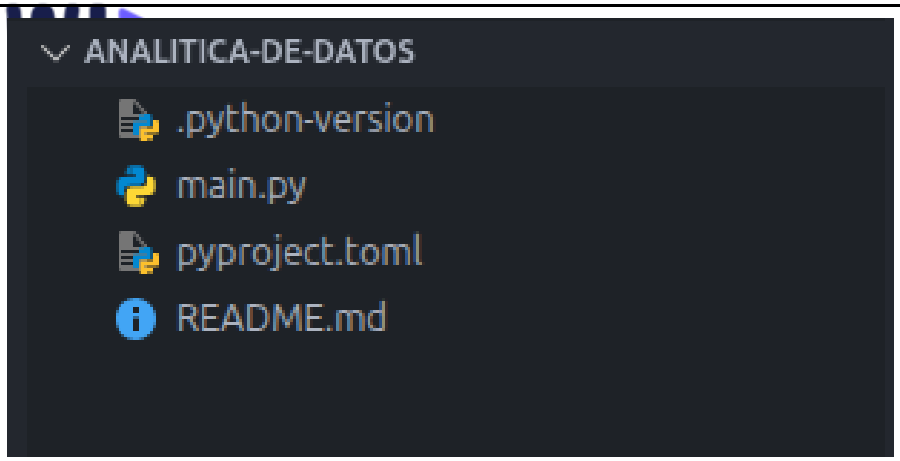
Aquí es donde entra UV.

### Inicialización de un proyecto de Python

Nos movemos a la carpeta donde queremos crear el codigo de Python y ejecutamos el siguiente comando:

```
jannergonzalez@RIWI-1699:~/Proyectos/riwi/ruta-avanzada/analitica-de-datos$ uv init
Initialized project `analitica-de-datos`
```

Al abrir la carpeta dentro de VS Code o nuestro IDE de preferencia debemos observar los siguientes archivos:




Perfecto, ya tenemos nuestro proyecto creado ahora vamos a lo siguiente, generar nuestro environment y activarlo para empezar a trabajar:

```
jannergonzalez@RIWI-1699:~/Proyectos/riwi/ruta-avanzada/analitica-de-datos$ uv venv .venv
Using CPython 3.12.13
Creating virtual environment at: .venv
Activate with: source .venv/bin/activate
jannergonzalez@RIWI-1699:~/Proyectos/riwi/ruta-avanzada/analitica-de-datos$ source .venv/bin/activate
(.venv) jannergonzalez@RIWI-1699:~/Proyectos/riwi/ruta-avanzada/analitica-de-datos$
```

Listo, con eso tenemos configurado tanto nuestro proyecto, como nuestro environment, tal como se hace en el mundo profesional.

## Instalación de Pandas

Ahora con UV Astral vamos a agregar pandas a nuestro proyecto y espacio de trabajo

```
(.venv) jannergonzalez@RIWI-1699:~/Proyectos/riwi/ruta-avanzada/analitica-de-datos$ uv add pandas
Resolved 6 packages in 0.72ms
Installed 4 packages in 16ms
 + numpy==2.4.3
 + pandas==3.0.1
 + python-dateutil==2.9.0.post0
 + six==1.17.0
```

Para verificar que tenemos configurado PANDAS dentro de nuestro proyecto vamos al archivo pyproject.toml y vemos que en dependecias ya se encuentre pandas:

```toml
[project]
name = "analitica-de-datos"
version = "0.1.0"
description = "Add your description here
readme = "README.md"
requires-python = ">=3.12"
dependencies = [
    "pandas>=3.0.1",
]
```

## De estructuras básicas a DataFrames: el cambio clave

Hasta ahora, trabajaste con estructuras como listas y diccionarios. Estas estructuras te permitieron representar datos, pero no están diseñadas para análisis a gran escala.
Por ejemplo, un dataset en Python puro podría verse así:

```python
datos = [
  {"cliente": "Juan", "ventas": 100},
   {"cliente": "Ana", "ventas": 200}
]
```

Esto funciona, pero es limitado. Si quieres filtrar, agrupar o transformar datos, necesitas múltiples pasos.
Pandas introduce una estructura completamente diferente: el DataFrame.

Un DataFrame es una tabla con filas y columnas, donde cada columna representa una variable y cada fila un registro. Es, en esencia, la forma en la que los datos existen en la mayoría de entornos analíticos.
Cuando conviertes tus datos a un DataFrame, estás dando el paso de trabajar con datos de forma manual a trabajar con datos de forma estructurada.
import pandas as pd

```python
# Importas para usar pandas con el alias de pd
import pandas as pd

# Forma antigua con array de diccionarios
datos = [
  {"cliente": "Juan", "ventas": 100},
   {"cliente": "Ana", "ventas": 200}
]

# Forma de convertir esos datos a un Dataframe
df = pd.DataFrame(datos)
```

Aquí ya no estás trabajando con listas, estás trabajando con una tabla. Y eso cambia completamente lo que puedes hacer.

## ¿Qué puedes hacer con Pandas?

Una vez tienes un DataFrame, empiezas a notar la diferencia inmediatamente.

Operaciones que antes requerían múltiples pasos ahora se pueden hacer de forma directa.

Puedes filtrar datos sin loops, crear nuevas columnas sin estructuras intermedias, agrupar información en una sola línea y combinar datasets de forma sencilla.

Esto significa que puedes enfocarte en lo importante:

**el análisis y no la implementación.**

```python
# Importas para usar pandas con el alias de pd
import pandas as pd

# Forma antigua con array de diccionarios
datos = [
    {"cliente": "Juan", "ventas": 100},
    {"cliente": "Ana", "ventas": 200}
]

# Crear dataset de clientes
df_clientes = pd.DataFrame(datos)

# Cargar un dataset desde un archivo CSV
df = pd.read_csv("ventas.csv")

# Filtrar información
df_filtrado = df[df["ventas"] > 150]

# Crear nuevas variables
df["ventas_doble"] = df["ventas"] * 2

# Agrupar datos para obtener métricas
resultado = df.groupby("cliente")["ventas"].sum()

# Combinar información de diferentes fuentes
df_completo = df.merge(df_clientes, on="cliente")
```

Cada una de estas operaciones representa algo que ya sabes hacer, pero ahora de una forma mucho más eficiente.

### El verdadero cambio: cómo piensas los datos

Este es el punto más importante de la lección.

Con Python puro, pensabas en términos de instrucciones:

- recorrer
- validar

- almacenar

Con Pandas, empiezas a pensar en términos de análisis:

- seleccionar
- transformar
- agrupar
- combinar

Este cambio de mentalidad es lo que realmente te acerca al rol de analista.

**Conclusiones del tema 2:**

En esta lección diste uno de los saltos más importantes en tu proceso de aprendizaje.

Pasaste de trabajar con datos de forma manual a trabajar con estructuras diseñadas específicamente para análisis. Entendiste que Pandas no solo simplifica el código, sino que transforma la forma en la que abordas los datos.

A partir de ahora, no se trata de escribir más código, sino de escribir mejor código y pensar mejor el análisis.

**Bibliografía:**

Escribe la bibliografía en normas APA 7ma ediciòn, recuerda hacer uso de https://www.bibguru.com/es/

| **Nombre del tema 3:** | Matplotlib |
|---|---|
| **Desarrollo del tema 3:** | |

## ¡Hola coder, ahora vamos a hacer visibles los datos!

Hasta este punto ya lograste algo muy importante. No solo sabes cargar datos, sino también explorarlos, limpiarlos, transformarlos y analizarlos utilizando Pandas.

Ya puedes responder preguntas como:

- ¿cuánto vendí por cliente?
- ¿qué producto tiene mejor desempeño?
- ¿cómo se comportan los datos en general?

Sin embargo, aquí aparece una nueva realidad.

Aunque ya tienes respuestas, muchas veces están en forma de tablas o números. Y aunque son correctos, no siempre son fáciles de interpretar.

En el mundo real, no basta con calcular resultados.

**Un analista debe ser capaz de comunicar lo que encontró.**

Y aquí es donde entra la visualización.

# Instalación y primeros pasos

Lo primero que debemos realizar es agregar la libreria de Matplotlib a nuestro proyecto de python tal como vemos en el siguiente ejemplo:

```
(.venv) jannergonzalez@RIWI-1699:~/Proyectos/riwi/ruta-avanzada/analitica-de-datos$ uv add matplotlib
Resolved 14 packages in 26.62s
Prepared 8 packages in 51.90s
Installed 8 packages in 22ms
 + contourpy==1.3.3
 + cycler==0.12.1
 + fonttools==4.62.1
 + kiwisolver==1.5.0
 + matplotlib==3.10.8
 + packaging==26.0
 + pillow==12.1.1
 + pyparsing==3.3.2
```

Ahora con el ejercicio de la lección de pandas vamos agregar un grafico y tomemos de referencia el codigo a continuación:

```python
# Importar para usar pandas con el alias de pd
# Importar matplotlib con el alias de plt
import pandas as pd
import matplotlib.pyplot as plt

# Cargar un dataset desde un archivo CSV
df = pd.read_csv("ventas.csv")

# Filtrar información
df_filtrado = df[df["ventas"] > 150]

# Crear nuevas variables
df["ventas_doble"] = df["ventas"] * 2

# Agrupar datos para obtener métricas
resultado = df.groupby("cliente")["ventas"].sum()

# Visualizar datos con Matplotlib
resultado.plot(kind="bar")
```

Ahora veamos otros tipos de graficos que tiene Matploitlib con los que podemos trabajar

```python
import pandas as pd
import matplotlib.pyplot as plt

# 1. BARRAS - Comparar categorías
ciudades = ['Bogotá', 'Medellín', 'Cali']
ventas = [1200, 950, 800]
df = pd.DataFrame({'Ciudad': ciudades, 'Ventas': ventas})
df.plot(kind='bar', x='Ciudad', y='Ventas')
plt.title("Ventas por ciudad")
plt.show()

# 2. LÍNEAS - Evolución tiempo
meses = ['Ene', 'Feb', 'Mar']
ventas = [100, 150, 200]
df = pd.DataFrame({'Mes': meses, 'Ventas': ventas})
df.plot(x='Mes', y='Ventas', kind='line')
plt.title("Ventas por mes")
plt.show()

# 3. HISTOGRAMA - Distribución
ventas = [100, 200, 150, 300, 250, 200, 100]
df = pd.DataFrame({'Ventas': ventas})
df['Ventas'].plot(kind='hist')
plt.title("Distribución ventas")
plt.show()

# 4. SCATTER - Relación variables
df = pd.DataFrame({'Clientes': [10, 20, 30, 40], 'Ventas': [100, 200, 300, 400]})
df.plot(x='Clientes', y='Ventas', kind='scatter')
plt.title("Clientes vs Ventas")
plt.show()
```

Ahora, coder, te invito a que sigas explorando más de estas librerías para que profundices tu conocimiento, averiguando que otras transformaciones, agregaciones o agrupaciones podemos hacer con pandas y luego como podemos graficas todos esos datos con matploitlib.

**Conclusiones del tema 3:**

En esta lección diste otro salto importante.

Pasaste de analizar datos a comunicarlos. Entendiste que el valor no está solo en calcular métricas, sino en hacerlas entendibles para otros.

Pandas te permite trabajar con datos. Matplotlib te permite explicarlos.